

DL Midsem Notes — Section 9

Regularization and Bias-Variance Tradeoff

1 Theory

1.1 Bias, Variance, Underfitting, Overfitting

Bias: error from a model being too simple to capture the true pattern (systematically wrong regardless of training data) → **underfitting**, high train *and* test error. **Variance:** error from a model being too sensitive to the specific training data it happened to see (different training sets give wildly different fitted models) → **overfitting**, low train error but high test error. Good models sit at a sweet spot minimizing both simultaneously — that’s rarely possible to push both to zero at once, hence ”tradeoff.”

1.2 Bias-Variance Decomposition (Setup)

Let $y = f(x) + \varepsilon$ (true function plus noise, $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$), and let $\hat{f}(x)$ be our model’s prediction (a random variable because it depends on the random training set D). The expected squared error at a point x , averaged over draws of D and the noise ε , decomposes as (full derivation in Q2):

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

1.3 Regularization Techniques

L2 (weight decay): add $\lambda\|w\|_2^2$ to the loss; penalizes large weights smoothly, shrinking all weights toward (but not exactly to) zero — reduces variance by discouraging the model from fitting noise with sharply-tuned large weights.

L1: add $\lambda\|w\|_1$; unlike L2 this drives many weights exactly to 0 (sparsity), effectively performing feature selection.

Early stopping: monitor validation loss during training; stop when it starts rising even as training loss keeps falling — that divergence point is exactly where the model starts memorizing noise instead of generalizing.

Dataset augmentation: synthesize more training examples (crops, flips, noise, etc.) so the model sees more variation and can’t as easily memorize any one exact example — effectively enlarges the dataset to reduce variance.

Dropout: during training, randomly zero out each neuron’s output with probability p (independently, per forward pass); forces the network to not rely on any single neuron/co-adapted group of neurons, acting like training an ensemble of thinner sub-networks that share weights.

2 Practice Questions

Q1 (PYQ-style, CS6510-1 Q3). Explain three regularization methods used to overcome overfitting of a complex model.

Q2 (PYQ-style, CS6510-3 Q3a / CS6510-4 Q3 / CS6510-6 Q3a — asked 3 times, high priority). Explain the bias-variance tradeoff with an appropriate diagram. Derive the equation for predicted error in terms of bias and variance.

Q3 (PYQ-style, CS6510-2 Q4, adapted). Sketch the typical early-stopping picture: what goes on each axis, and how do the training and validation curves differ in shape? Where exactly do you stop?

Q4 (PYQ-style, CS6510-5 Q6c). Mention at least 8 forms of regularization used in deep learning.

Q5 (concept check). L1 vs L2 regularization: what is the key practical difference in the resulting weights, and when would you prefer L1?

3 Answers

A1. L2 regularization: adds $\lambda\|w\|_2^2$ to the loss, shrinking all weights smoothly toward zero and discouraging the model from relying on any single feature too heavily — reduces variance. **Dropout:** randomly zeroes neurons during training, preventing co-adaptation and approximating an ensemble of sub-networks, which generalizes better than any single overfit network. **Early stopping:** halts training at the point where validation loss starts increasing even though training loss keeps falling, stopping before the model starts memorizing training-set-specific noise.

A2. Diagram (described): x-axis = model complexity (or training epochs), y-axis = error. Bias² is a decreasing curve (complex models fit the true pattern better); Variance is an increasing curve (complex models are more sensitive to the specific training set); total expected error is U-shaped, their sum plus a constant irreducible-noise floor, minimized at some intermediate complexity.

Derivation. Let $y = f(x) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of $\hat{f}$. Write $\bar{f} = \mathbb{E}[\hat{f}(x)]$ (expectation over training sets D). Then:

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f + \varepsilon - \hat{f})^2] = \mathbb{E}[(f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon]^2$$

Expand the square; $f - \bar{f}$ is a constant (no randomness), ε has mean 0 and is independent of everything else, and $\mathbb{E}[\bar{f} - \hat{f}] = 0$ by definition of $\bar{f}$, so every cross term vanishes on taking expectation:

$$= (f - \bar{f})^2 + \mathbb{E}[(\bar{f} - \hat{f})^2] + \mathbb{E}[\varepsilon^2] = \underbrace{(\bar{f} - f)^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f})}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

A3. x-axis: training epochs/iterations. y-axis: loss/error. Training loss curve: monotonically decreasing throughout (the model keeps fitting the training set better). Validation loss curve: decreases initially alongside training loss, reaches a minimum, then *increases* even as training loss keeps dropping — that rising gap is overfitting setting in. Stop training exactly at the validation curve's minimum (before it starts climbing), even though training loss would keep improving if you continued.

A4. L1 regularization, L2 regularization (weight decay), dropout, early stopping, dataset augmentation, batch normalization (has a regularizing side-effect), noise injection (to inputs or weights during training), parameter sharing/tying (e.g. convolutional weight sharing), and ensembling/bagging (train multiple models, average predictions) — any 8 of these.

A5. L2 shrinks weights smoothly toward zero but essentially never sets them exactly to zero; L1 tends to push many weights to exactly zero, producing a sparse weight vector. Prefer L1 when you want implicit feature selection — a sparse model that has effectively "chosen" a subset of the most useful inputs/features — rather than just a smaller-magnitude dense model.